

DC Motor, No Load Test, data processing

Packages

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np
import seaborn as sns
import os
import json
import warnings
```

Configuration

```
In [ ]: # Folder name
CSV_folder_name = "CSV_files"
figure_folder_name = "figures"
json_folder_name = "json"

# Set Seaborn style
sns.set_style("whitegrid")

# Option to save data and figures ( True = Save data and figures, False = Don't save)
SAVE = False

# Ignore warnings
warnings.filterwarnings("ignore", message="invalid value encountered in subtract")
```

Load Data Set

```
In [ ]: # Load data from CSV file
file_name = "motor2.csv" # CSV name
file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
raw_df = pd.read_csv(file_path) # Read CSV file.
voltage_step = 0.5 # Voltage step, 1 = 1V, 2V, 3V, 4V etc...

# Load data from json file
motor_file = "motor_data.json" # CSV name
file_path = os.path.join(json_folder_name, motor_file) # Generate file path
with open(file_path, 'r') as json_file:
    motor_data = json.load(json_file)
```

View Data

We start of by viewing the data and calculating the RPM. This can be done with the following formula:

$$V_r = \dot{\theta} = \frac{d}{dt}(\theta) \cdot \frac{60}{360} \quad (1)$$

```

In [ ]: # Select example sample from data set
sample = 3
data_type = 0 # (Angle = 0, V_bus = 1, V_shunt = 2, current = 3)

# Create new dataframe for rotor data, containing rotor position and time
rotor_data = pd.concat([raw_df.iloc[:, (5*sample + data_type)], raw_df.iloc[:, (5*sample + data_type + 1)]], axis=1)
rotor_data.columns = ['rotor_position', 'Time']

# Calculate motor RPM from raw data
rotor_data['RPM'] = rotor_data['rotor_position'].diff() / rotor_data['Time'].diff()

# Plotting
## Create Figure
plt.figure(figsize=(10, 6))

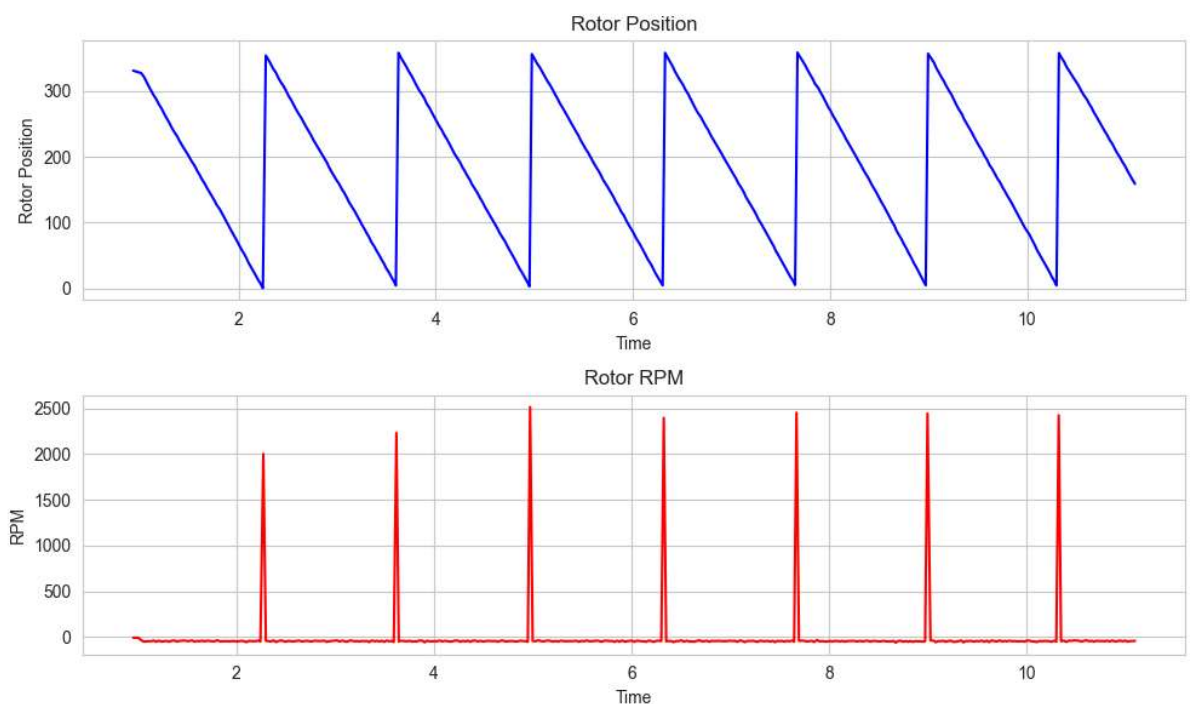
## Rotor position plot
plt.subplot(2, 1, 1)
plt.plot(rotor_data['Time'], rotor_data['rotor_position'], color='blue')
plt.title("Rotor Position")
plt.xlabel('Time')
plt.ylabel('Rotor Position')

## Motor RPM plot
plt.subplot(2, 1, 2)
plt.plot(rotor_data['Time'][1:], rotor_data['RPM'][1:], color='red') # Omit first
plt.title("Rotor RPM")
plt.xlabel('Time')
plt.ylabel('RPM')

# Save figure
file_name = "raw_rotor_pos_and_RPM.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

## Show plot
plt.tight_layout()
plt.show()

```



From the plot above, we see we counter two issues with the data:

- The rotor position goes from 0-359 degrees, resetting every 360 degrees.
- Every time the rotor position resets, the estimated velocity goes to the moon.
- The motor spins counter clock wise (CCW).

In order to improve the angular velocity estimate, we can do two things:

- Invert the rotor position data, from 359-0 to 0-359 degrees.
- Remove all negative values from the angular velocity estimate, since we know the motor spins in one direction at a constant speed.

```
In [ ]: # Invert data
rotor_data['rotor_position'] = 359 - rotor_data['rotor_position']

# Calculate motor RPM from raw data
rotor_data['RPM'] = rotor_data['rotor_position'].diff() / rotor_data['Time'].diff()

# Remove negative values
rotor_data = rotor_data[rotor_data['RPM'] >= 0]

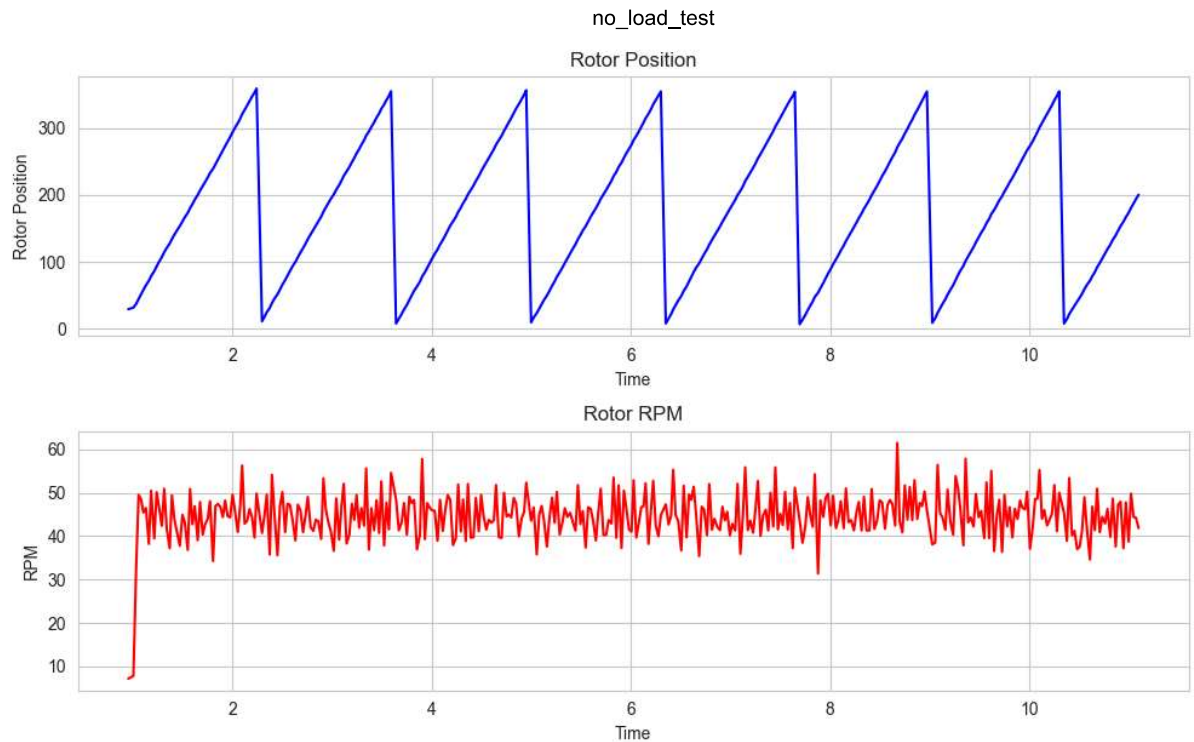
# Plotting
## Create Figure
plt.figure(figsize=(10, 6))

## Rotor position plot
plt.subplot(2, 1, 1)
plt.plot(rotor_data['Time'], rotor_data['rotor_position'], color='blue')
plt.title("Rotor Position")
plt.xlabel('Time')
plt.ylabel('Rotor Position')

## Motor RPM plot
plt.subplot(2, 1, 2)
plt.plot(rotor_data['Time'], rotor_data['RPM'], color='red')
plt.title("Rotor RPM")
plt.xlabel('Time')
plt.ylabel('RPM')
plt.tight_layout()

# Save figure
file_name = "corrected_rotor_pos_and_RPM.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

## Show plot
plt.show()
```



Now the data is looking a lot more reasonable. But we can still see that the speed data is containing some artifacts. We can filter this out with outlier detection, and interpolate the missing values.

```
In [ ]: # Calculate z-scores for each value in the RPM column
z_scores = stats.zscore(rotor_data['RPM'])

# Define a threshold for identifying outliers (e.g., 3 standard deviations from the
threshold = 3

# Find the indices of outliers
outlier_indices = np.abs(z_scores) > threshold

# Remove outliers from the DataFrame
cleaned_rotor_data = rotor_data[~outlier_indices]

# Calculate mean RPM before removing outliers
mean_rpm_before = rotor_data['RPM'].mean()

# Calculate mean RPM after removing outliers
mean_rpm_after = cleaned_rotor_data['RPM'].mean()

# Plot 'RPM' column before removing outliers
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(rotor_data['Time'], rotor_data['RPM'], color='blue')
plt.axhline(y=mean_rpm_before, color='green', linestyle='--', label=f'Mean RPM: {mean_rpm_before}')
plt.title('Before Removing Outliers')
plt.xlabel('Time')
plt.ylabel('RPM')
plt.legend()

# Plot 'RPM' column after removing outliers
plt.subplot(2, 1, 2)
plt.plot(cleaned_rotor_data['Time'], cleaned_rotor_data['RPM'], color='red')
plt.axhline(y=mean_rpm_after, color='green', linestyle='--', label=f'Mean RPM: {mean_rpm_after}')
plt.title('After Removing Outliers')
plt.xlabel('Time')
plt.ylabel('RPM')
```

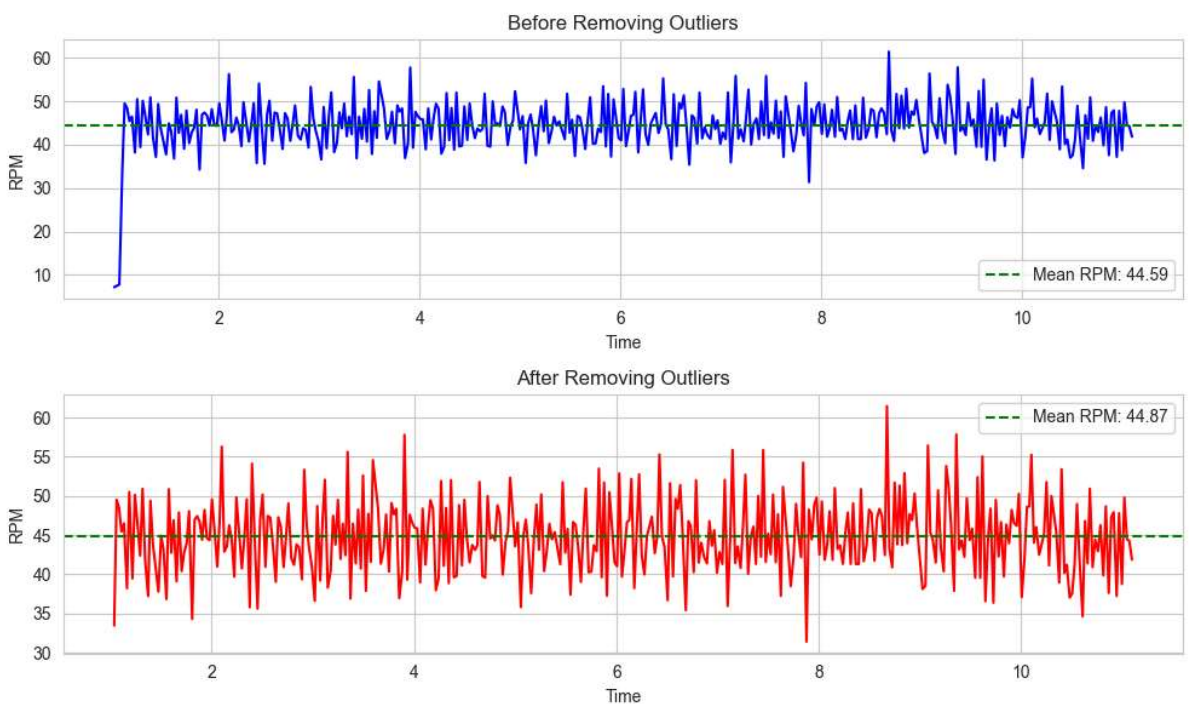
```
plt.legend()
plt.tight_layout()

# Save figure
file_name = "filtered_and_unfiltered_RPM.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

# Show plot
plt.show()

# Update RPM in rotor_data
rotor_data['RPM'] = cleaned_rotor_data['RPM']

# Print mean RPM before and after outlier removal
print(f"The mean RPM before outlier removal is: {mean_rpm_before:.4f} RPM")
print(f"The mean RPM after outlier removal is: {mean_rpm_after:.4f} RPM")
print(f"This amounts to a difference of: {abs(mean_rpm_before - mean_rpm_after):.4f} RPM")
```



The mean RPM before outlier removal is: 44.5858 RPM

The mean RPM after outlier removal is: 44.8719 RPM

This amounts to a difference of: 0.2861 RPM

With this data filtering, we will assume that we have a decent data set, and can apply this to the entirety of the data.

```
In [ ]: RPM_df = pd.DataFrame()

# Find length of columns
num_columns = len(raw_df.columns)

# Iterating through every fifth column
for sample in range(0, num_columns, 5):
    # Create new dataframe for rotor data, containing rotor position and time
    rotor_data = pd.concat([raw_df.iloc[:, (sample)], raw_df.iloc[:, (sample + 4)]])
    rotor_data.columns = ['rotor_position', 'Time']

## Invert data
rotor_data['rotor_position'] = 359 - rotor_data['rotor_position']
```

```
# Calculate motor RPM from raw data
rotor_data['RPM'] = rotor_data['rotor_position'].diff() / rotor_data['Time'].diff()

# Remove negative values
rotor_data = rotor_data[rotor_data['RPM'] >= 0]

# Calculate z-scores for each value in the 'RPM' column
z_scores = stats.zscore(rotor_data['RPM'])

# Define a threshold for identifying outliers (e.g., 3 standard deviations from
threshold = 3

# Find the indices of outliers
outlier_indices = np.abs(z_scores) > threshold

# Remove outliers from the DataFrame
cleaned_rotor_data = rotor_data[~outlier_indices]

# Add to RPM dataframe
RPM_df[f'{(sample/5) * voltage_step}V'] = cleaned_rotor_data['RPM']

# Final data clean up
# Replace "inf" values with NaN
RPM_df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Replace NaN values in each column with interpolation
RPM_df = RPM_df.interpolate()

# Remove first rows with NaN values.
nan_index = RPM_df.index[RPM_df.isnull().any(axis=1)][0]
RPM_df = RPM_df.iloc[nan_index + 2:]

# Remove non-zero values from first column, we know the speed of the motor was zero
RPM_df.iloc[:, :3] = 0

# Reset index to start from zero
RPM_df = RPM_df.reset_index(drop=True)

# Display Table
rounded_df = RPM_df.round(1)
rounded_df
```

Out[]:

	0.0V	0.5V	1.0V	1.5V	2.0V	2.5V	3.0V	3.5V	4.0V	4.5V	...	12.5V	13.0V	13.5V	14.0V
0	0.0	0.0	0.0	49.5	81.7	98.8	118.2	193.8	203.9	286.2	...	893.9	946.9	931.2	744.5
1	0.0	0.0	0.0	48.5	85.0	125.3	148.8	154.0	203.4	252.6	...	958.9	931.9	835.8	825.6
2	0.0	0.0	0.0	45.4	71.2	105.4	137.4	172.9	268.4	231.2	...	652.9	688.8	782.5	812.5
3	0.0	0.0	0.0	38.2	70.2	107.9	136.2	194.4	230.4	253.1	...	710.0	792.3	768.1	812.5
4	0.0	0.0	0.0	50.5	88.9	112.7	120.6	157.8	202.8	253.1	...	877.9	920.0	1056.4	912.5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
343	0.0	0.0	0.0	47.9	98.1	97.9	152.9	150.3	204.6	244.6	...	818.3	698.9	745.3	1012.5
344	0.0	0.0	0.0	37.2	71.4	103.9	178.5	158.1	236.5	231.7	...	874.1	845.6	790.4	1012.5
345	0.0	0.0	0.0	47.8	73.2	133.0	162.2	165.9	195.7	218.8	...	687.1	817.0	835.6	1012.5
346	0.0	0.0	0.0	49.8	76.1	129.0	145.8	168.3	216.8	246.0	...	765.6	788.5	830.6	1112.5
347	0.0	0.0	0.0	44.5	70.1	118.9	136.7	193.8	228.0	249.7	...	775.9	788.5	825.6	1112.5

348 rows × 35 columns

Perform Linear Regression on the data set to test data

```
In [ ]: # Initialize empty numpy arrays for x and y
x = np.array([])
y = np.array([])

n = 0
# Loop through each column in the DataFrame
for column in RPM_df.columns:
    # Add values from the column to y array
    x = np.append(x, RPM_df[column].values)

    # Add an array of 2s to x array (assuming the same length as y)
    y = np.append(y, np.full_like(RPM_df[column].values, n*0.5))
    n += 1

# Perform Linear regression
slope, intercept, _, _, stderr = stats.linregress(x, y)
print("Regression line: y(x) = {:.3f} + {:.4f}x.".format(intercept, abs(slope)))
print("Regression line: V(RPM) = {:.3f}V + {:.4f}*RPM.".format(intercept, abs(slope)))
# Calculate confidence interval
conf_interval = 1.96 * stderr # 95% confidence interval

# Create scatter plot of x and y values with seaborn
sns.scatterplot(x=x, y=y)

# Plot the regression line with confidence interval using seaborn
sns.lineplot(x=x, y=slope * x + intercept, color='red', label='Linear Regression',

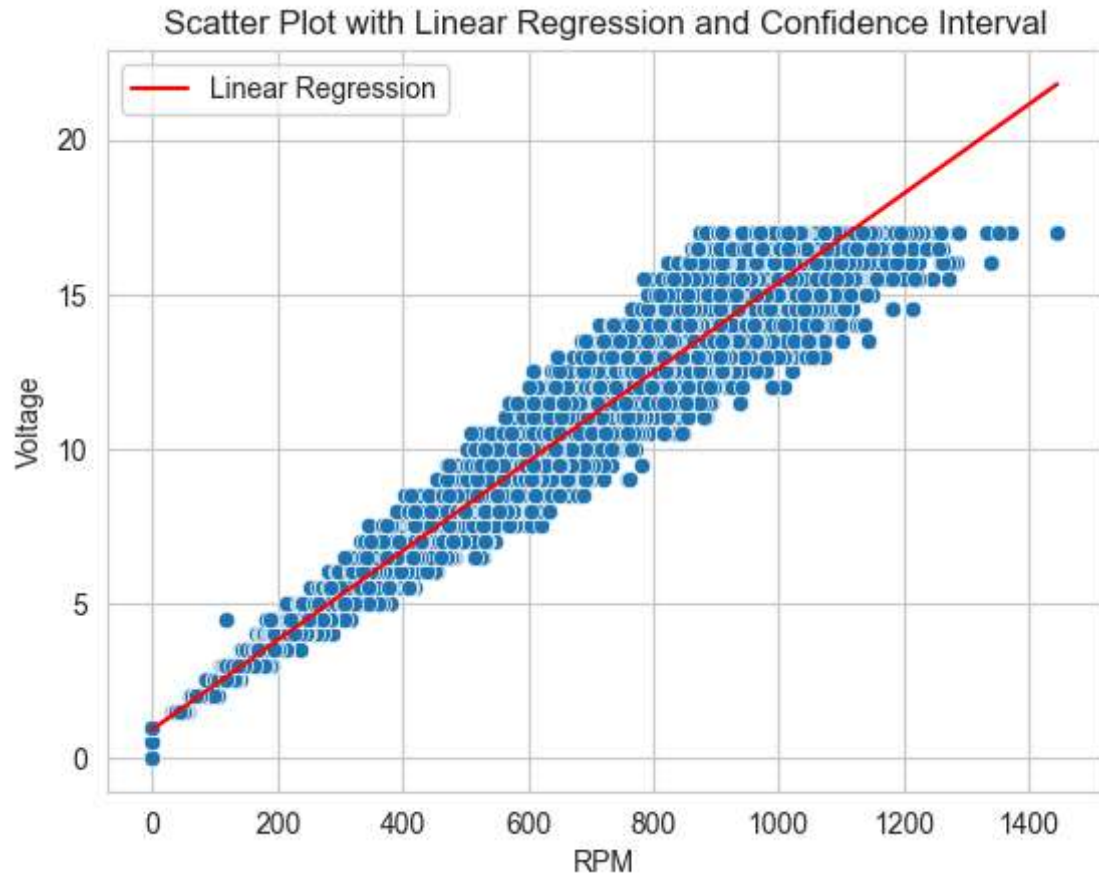
# Add labels and title
plt.xlabel('RPM')
plt.ylabel('Voltage')
plt.title('Scatter Plot with Linear Regression and Confidence Interval')
plt.legend() # Show Legend
```

```
# Save figure
file_name = "motor_constant_lin_reg.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

# Show the plot
plt.show()
```

Regression line: $y(x) = 0.905 + 0.0145x$.

Regression line: $V(\text{RPM}) = 0.905V + 0.0145 \cdot \text{RPM}$.



We need to calculate the back-EMF E , in order to get a realistic estimate for the motor constant. We can do this with the following formula:

$$E = K_m \dot{\theta} = (V - i_a \cdot R_a - V_{shunt}) \quad (2)$$

```
In [ ]: # Create an empty DataFrame for motor constant
K_motor_df = pd.DataFrame()

# Iterating through every fifth column
for sample in range(0, len(raw_df.columns), 5):
    # Import data
    ## Create temporary dataframe for motor data
    temp_df = pd.concat([raw_df.iloc[:, (sample + 4)], raw_df.iloc[:, sample], raw_df.iloc[:, sample+1], raw_df.iloc[:, sample+2]], axis=1)
    temp_df.columns = ['Time', 'rotor_position', 'current', 'V_shunt']

    ## Rescale data
    temp_df['rotor_position'] = 359 - temp_df['rotor_position'] # Reverse rotation
    temp_df['current'] /= 1000 # Convert from mA to A
    temp_df['V_shunt'] /= 1000 # Convert from mV to V

    # Calculate motor RPM from raw data
    temp_df['RPM'] = temp_df['rotor_position'].diff() / temp_df['Time'].diff()
```

```

temp_df['RPM'] *= (60 / 360) # Convert from deg/s to RPM

# Calculate BEMF [E]
## Add bus voltage
temp_df['V'] = voltage_step * (sample / 5) * np.ones(len(raw_df))

## Calculate IR
temp_df['V_IR'] = (temp_df['current']) * motor_data['R_a'] # Calculate IR.

## Calculate back EMF (BEMF)(E)
temp_df['V_E'] = temp_df['V'] - temp_df['V_IR'] - (temp_df['V_shunt'])

# Clean up data
## Remove negative values
temp_df = temp_df[temp_df['RPM'] >= 0]

## Replace "inf" values with NaN
temp_df.replace([np.inf, -np.inf], np.nan, inplace=True)

## Remove values above 2000 RPM
temp_df = temp_df[temp_df['RPM'] <= 2000]

## Calculate z-scores for each value in the 'RPM' column
z_scores = stats.zscore(temp_df['RPM'])

## Define a threshold for identifying outliers (e.g., 3 standard deviations from
threshold = 3

## Find the indices of outliers
outlier_indices = np.abs(z_scores) > threshold

## Remove outliers from the DataFrame
temp_df = temp_df[~outlier_indices]

## Replace NaN values in each column with interpolation
temp_df = temp_df.interpolate()

# Append temp_df to K_motor_df
K_motor_df = pd.concat([K_motor_df, temp_df], ignore_index=True)

## Reset index to start from zero
K_motor_df = K_motor_df.reset_index(drop=True)

# Save data
if SAVE:
    file_name = "K_motor.csv" # CSV name
    file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
    K_motor_df.to_csv(file_path, index=False) # Save df

# Display Table
rounded_df = K_motor_df.round(1)
rounded_df

```

Out[]:

	Time	rotor_position	current	V_shunt	RPM	V	V_IR	V_E
0	1.0	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
1	1.0	212.5	-0.0	0.0	0.6	0.0	-0.0	0.0
2	1.0	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
3	1.0	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
4	1.1	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
...	...	...	...	...	...	...	...	...
10602	11.5	321.5	0.2	0.0	1209.9	17.0	0.9	16.1
10603	11.5	305.1	0.2	0.0	1192.7	17.0	1.0	15.9
10604	11.6	294.8	0.2	0.0	1206.9	17.0	1.1	15.9
10605	11.6	278.6	0.1	0.0	1013.7	17.0	0.6	16.4
10606	11.7	262.3	0.2	0.0	1209.0	17.0	1.1	15.9

10607 rows × 8 columns

Now with this new data, we can redo the linear regression.

In []:

```

# Extract x and y values
x = K_motor_df['RPM'].values
y = K_motor_df['V_E'].values

# Perform linear regression
slope, intercept, _, _, stderr = stats.linregress(x, y)
print("Regression line:  $y(x) = {:.3f} + {:.4f}x$ .".format(intercept, abs(slope)))
print("Regression line:  $V(RPM) = {:.3f}V + {:.4f} \cdot RPM$ .".format(intercept, abs(slope)))
print("Motor Constant:  $( {:.2f}mV / RPM )$ .".format(slope*1000))

# Calculate estimated values
K_motor_df['V_E_hat'] = slope * x + intercept

# Save data
if SAVE:
    file_name = "K_motor.csv" # CSV name
    file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
    K_motor_df.to_csv(file_path, index=False) # Save df

# Plot the regression line with confidence interval using seaborn
sns.lineplot(x=x, y=( K_motor_df['V_E_hat'].values ), color='red', label='Linear Regression')

# Create scatter plot of x and y values with seaborn
sns.scatterplot(x=x, y=y, label='Data points')

# Add Labels and title
plt.xlabel('RPM')
plt.ylabel('Voltage')
plt.title('Scatter Plot with Linear Regression')
plt.legend() # Show Legend

# Save figure
file_name = "motor_constant_w_o_IR.pdf" # Plot name
file_path = os.path.join(figure_folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

```

```

# Show the plot
plt.show()

# Save data
if SAVE:
    ## Generate file path
    file_path = os.path.join(json_folder_name, motor_file)

    ## Update K_V and K_M
    motor_data['K_V'] = slope
    motor_data['K_m'] = slope * ( (2*np.pi)/60)

    ## Save the updated dictionary to the JSON file
    with open(file_path, 'w') as json_file:
        json.dump(motor_data, json_file)

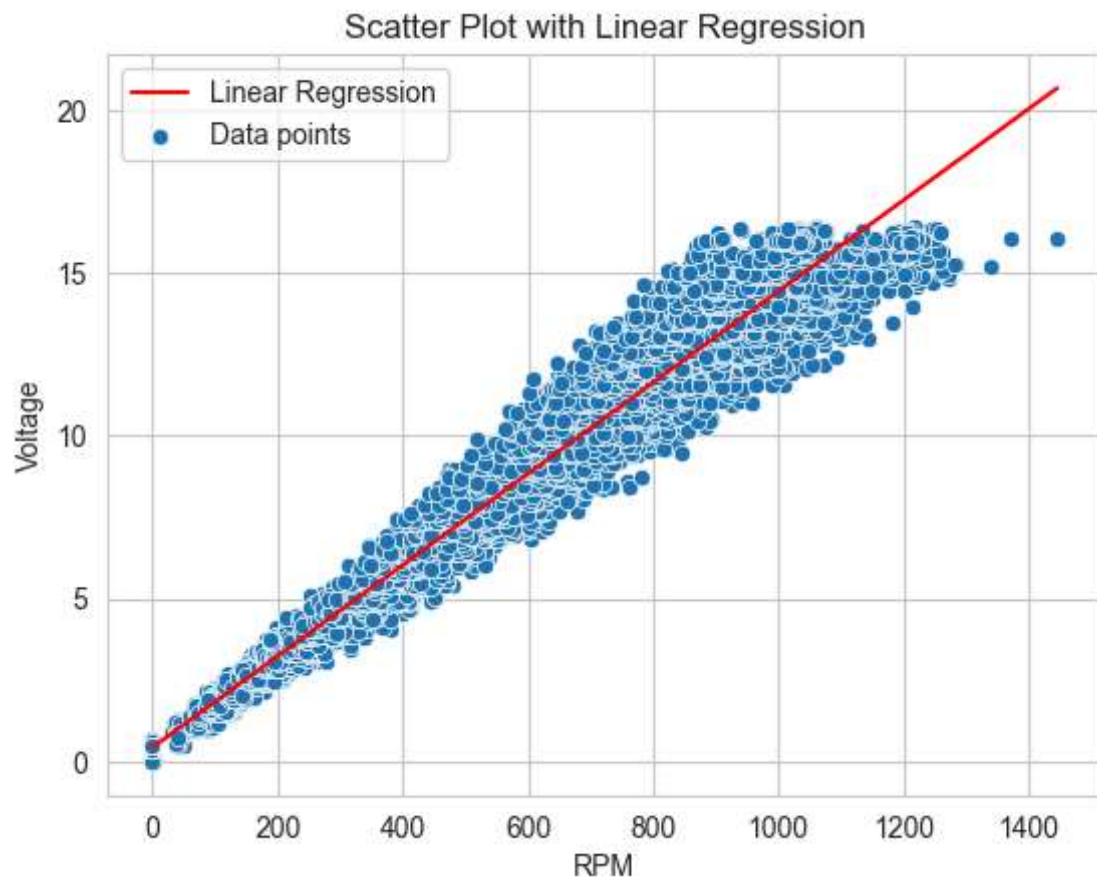
    ## Print new motor data
    print("New motor data:")
    print("R_a = {:.2f}".format(motor_data['R_a']))
    print("K_V = {:.4f}".format(motor_data['K_V']))
    print("K_m = {:.4f}".format(motor_data['K_m']))
    print("")

```

Regression line: $y(x) = 0.424 + 0.0140x$.

Regression line: $V(\text{RPM}) = 0.424\text{V} + 0.0140 \cdot \text{RPM}$.

Motor Constant: (14.01mV / RPM).



The data now clusters more evenly, which lines up with expectations. A comparison between the two is provided below:

```

In [ ]: # Initialize empty numpy arrays for x and y
x = np.array([])
y = np.array([])
x2 = K_motor_df['RPM'].values

```

```

y2 = K_motor_df['V_E'].values

n = 0
# Loop through each column in the DataFrame
for column in RPM_df.columns:
    # Add values from the column to y array
    x = np.append(x, RPM_df[column].values)

    # Add an array of 2s to x array
    y = np.append(y, np.full_like(RPM_df[column].values, n*0.5))
    n += 1

# Create a figure and two subplots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10))

# Plot the first subplot
sns.scatterplot(x=x, y=y, ax=ax1)
ax1.set_xlabel('RPM')
ax1.set_ylabel('Voltage')
ax1.set_title('Data points using bus voltage')
ax1.legend()

# Plot the second subplot
sns.scatterplot(x=x2, y=y2, ax=ax2)
ax2.set_xlabel('RPM')
ax2.set_ylabel('Voltage')
ax2.set_title('Data points using back-EMF.')

# Figure settings
plt.tight_layout()

# Save figure
file_name = "voltage_comparison.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

# Show the plot
plt.show()

```

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

